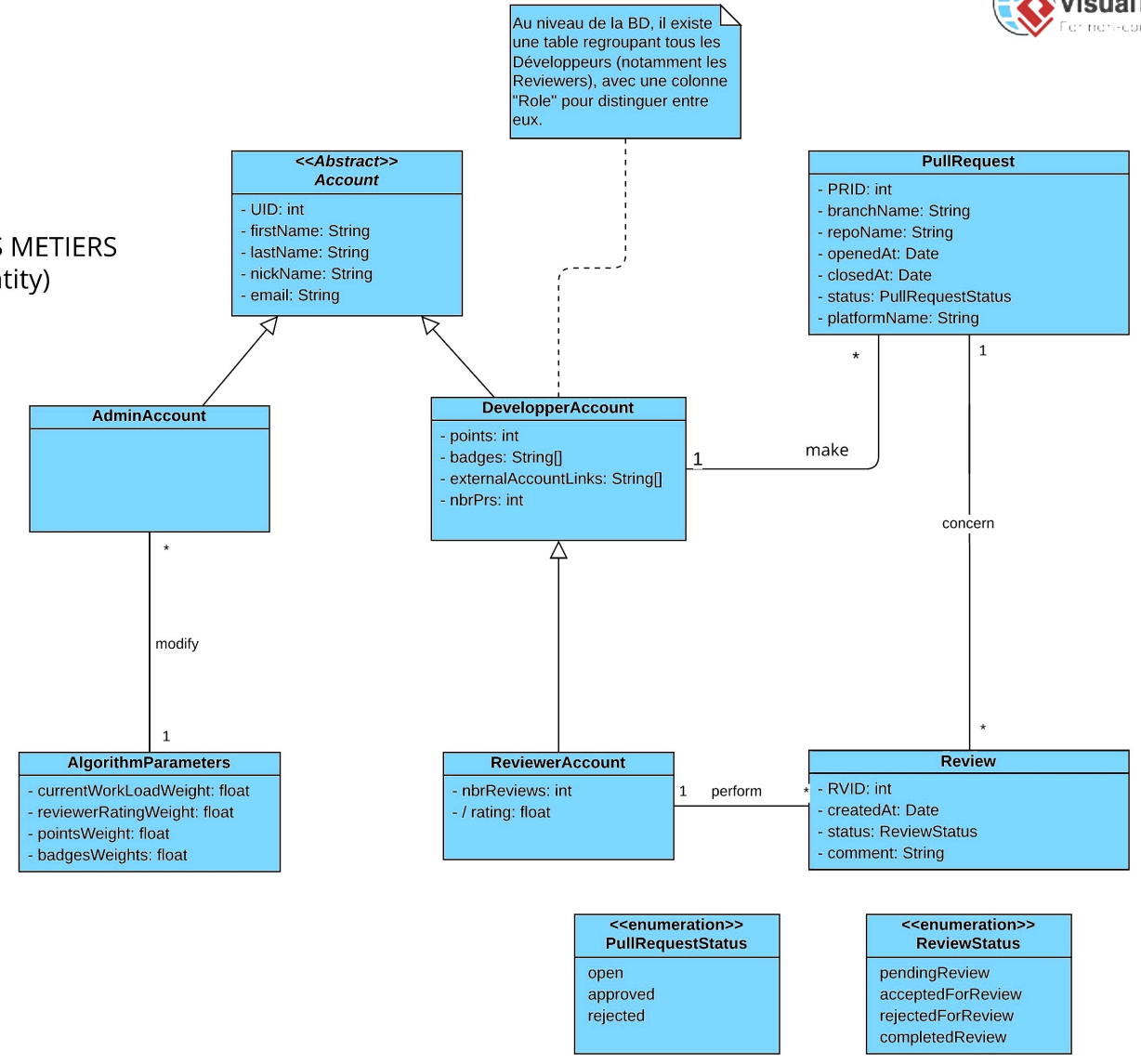
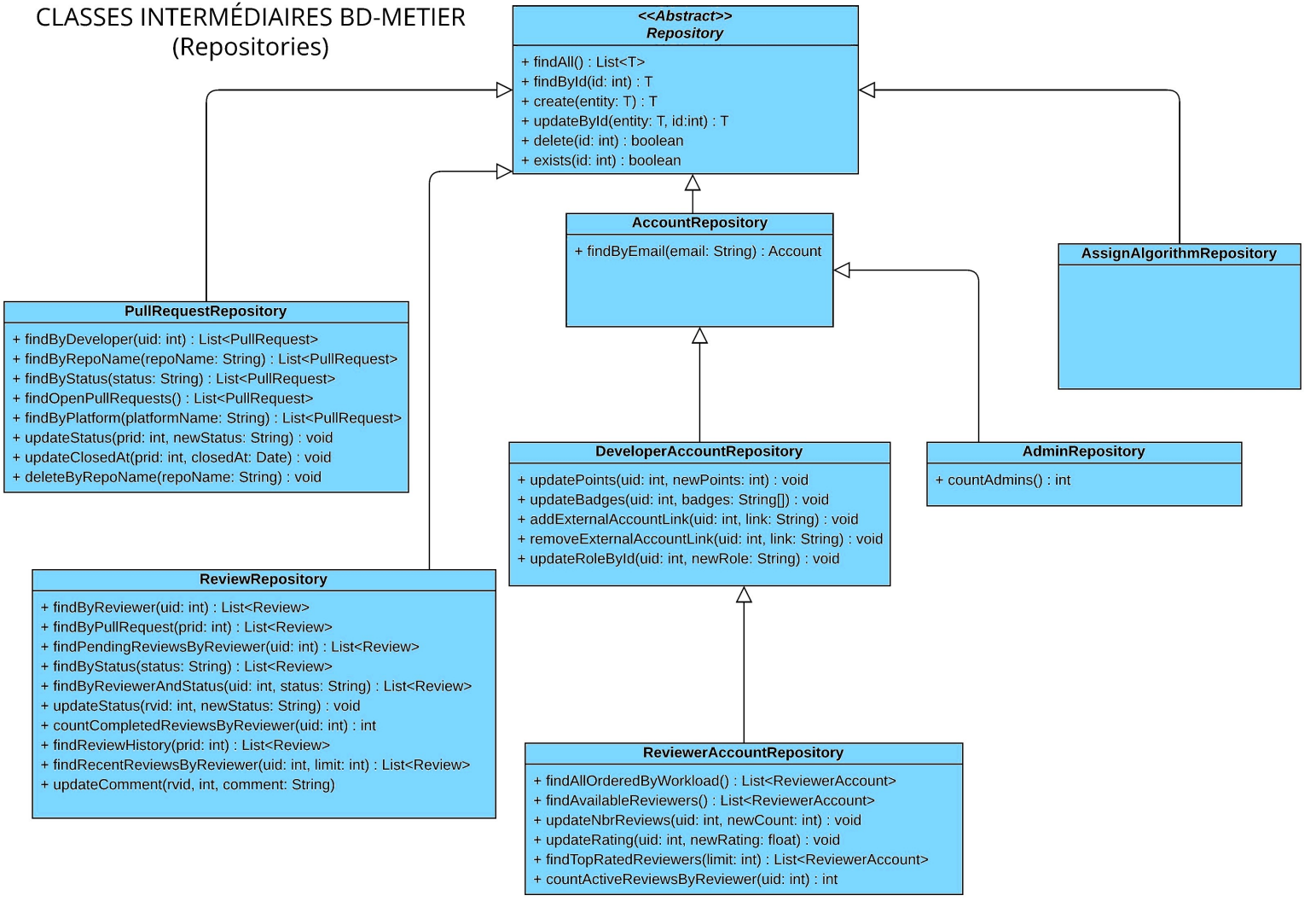
Made with Visual Paradigm
For non-commercial use

CLASSES METIERS (Entity)



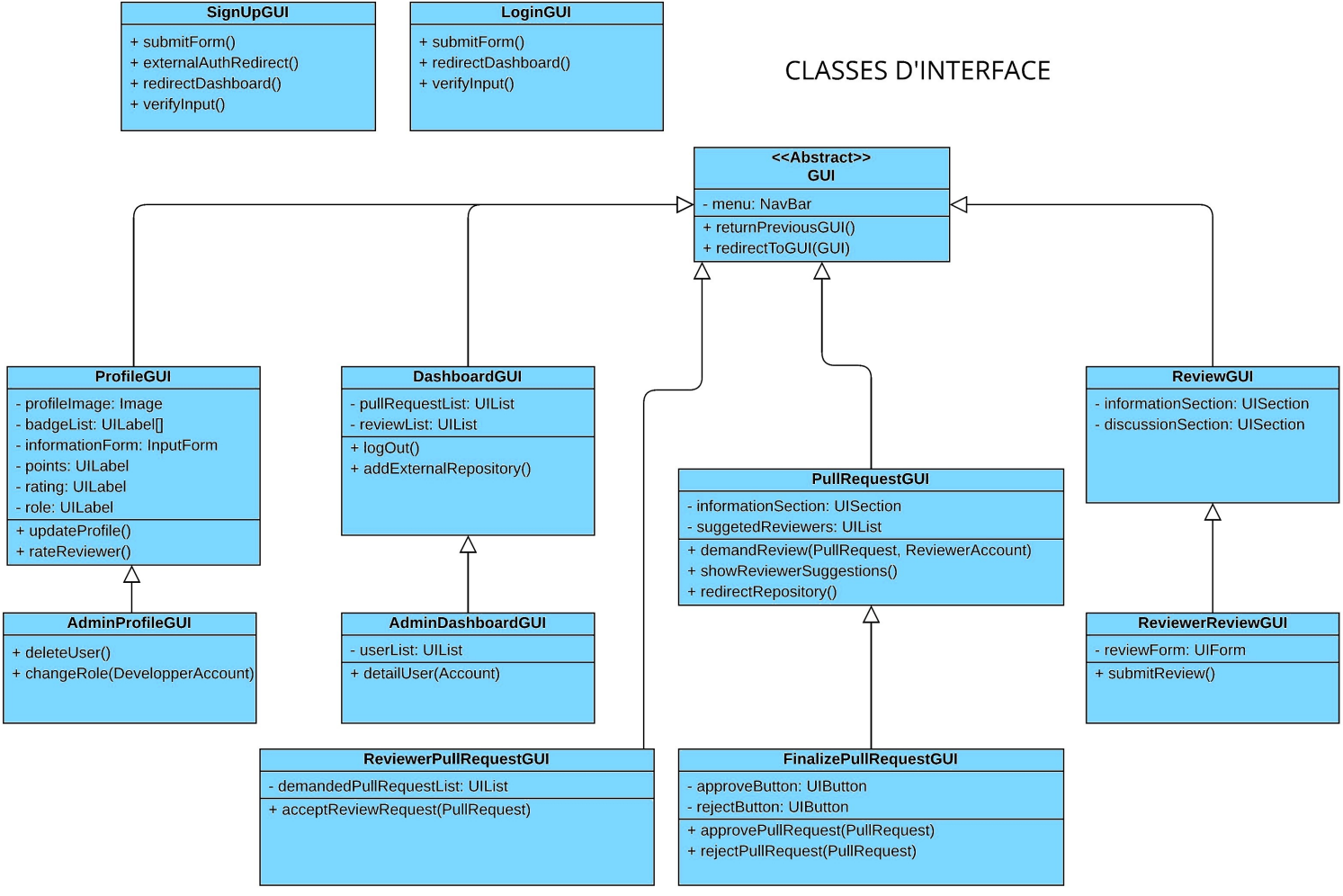
Made with Visual Paradigm
For non-commercial use

CLASSES INTERMÉDIAIRES BD-METIER (Repositories)



Made with Visual Paradigm
For non-commercial use

CLASSES D'INTERFACE



CLASSES DE CONTRÔLE
(Controller)

Made with Visual Paradigm
For non-commercial use

ReviewController

+ loadRequestedReviews(reviewer: ReviewerAccount) : List<Review>
+ getReviewDetails(rvid: int) : Review
+ submitReview(reviewId: int, comment: String) : void
+ acceptReviewRequest(pr: PullRequest) : void

PullRequestController

+ loadPullRequests(dev: DevelopperAccount) : List<PullRequest>
+ importRepository(dev: DevelopperAccount, repoName: String) : void
+ claimPullRequest(pr: PullRequest) : void
+ assignReviewers(pr: PullRequest, reviewers: List<ReviewerAccount>) : void
+ getPullRequestDetails(prid: int) : PullRequest
+ finalizePullRequest(prid: int, decision: String) : void

AttributionController

+ suggestReviewers(pr: PullRequest) : List<ReviewerAccount>
+ calculateScore(reviewer: ReviewerAccount, algo: AlgorithmParameters) : float
+ getCurrentAlgorithm() : AlgorithmParameters

AuthentificationController

+ login(email, password) : Account
+ signUp(accountData) : DevelopperAccount
+ linkExternalAccount(platformName, credentials) : boolean
+ logout() : void
+ verifyCredentials(email, password) : boolean

SynchronisationController

+ synchronizePRs(developer: DevelopperAccount) : List<PullRequest>
+ fetchExternalRepos(link: String) : List<String>
+ notifyExternalPlatform(pr: PullRequest, decision: String) : void

GamificationController

+ awardPoints(developper: DevelopperAccount, action: String) : void
+ checkBadgeConditions(reviewer: ReviewerAccount) : List<String>
+ grantBadge(reviewer: ReviewerAccount, badge: String) : void
+ updateRating(reviewer: ReviewerAccount, rating: float) : void

NotificationController

+ notifyReviewRequest(reviewer: ReviewerAccount, pr: PullRequest) : void
+ notifyPROwner(developer: DevelopperAccount, decision: String) : void
+ sendEmail(account: Account, subject: String, body: String) : void

DashboardController

+ loadDashboard(account: Account) : DashboardData
+ loadProfile(account: Account) : ProfileData
+ loadUserList() : List<Account>
+ updateProfile(account: Account, data: ProfileData) : void

AdministrationController

+ updateAlgorithmParameters(params: AssignAlgorithm) : void
+ deleteUser(account: Account) : void
+ changeRole(dev: DevelopperAccount, newRole: String) : void
+ manageUsers() : List<Account>

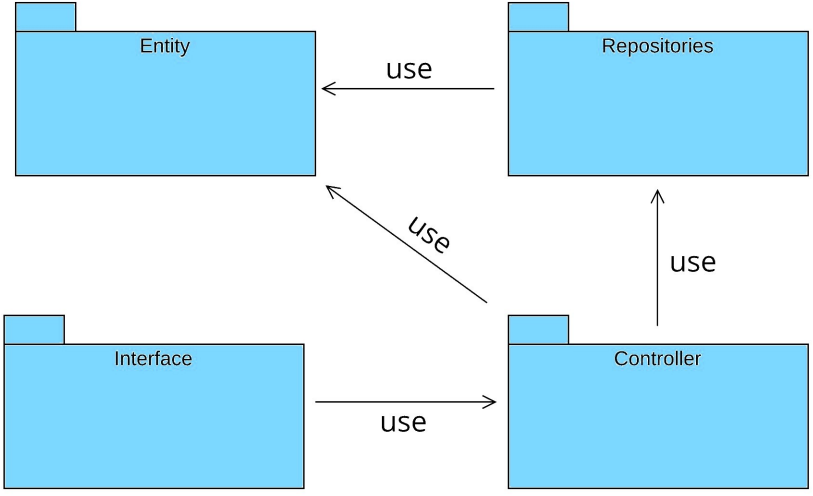



Diagramme de packages